Shell scripting

Write Bash scripts that fail loudly, run safely, parse JSON from APIs and replace repetitive manual work.

A MARGA printable pack. Free, offline, no sign-up. Learn online at marga.eknathalabs.com.

MARGA by EknathaLabs
marga.eknathalabs.com

Contents

1. Script basics (14 min)
2. Control flow and functions (14 min)
3. Text processing with grep, sed and awk (15 min)
4. Advanced Bash: arguments, arrays and here-documents (15 min)
5. Working with APIs and JSON using curl and jq (14 min)
6. Safe automation: strict mode, traps and scheduling (16 min)
7. Arrays, associative arrays and parameter expansion (13 min)
8. Building and testing shell tools (13 min)
9. Temporary files, locks and atomic writes (12 min)
10. Portable scripts, linting and style (11 min)
11. Debugging shell scripts (14 min)
12. Log analysis one-liners and pipelines (12 min)
Cheat sheet

Lesson 1. Script basics

14 minutes. Write your first reliable Bash script: shebang, variables, quoting, arguments, exit codes and command substitution.

Your team runs the same six commands after every deploy: check disk space, check the service, tail the log. Doing it by hand invites mistakes at 2 a.m. A script runs the same steps the same way every time and can be reviewed like code. Bash is already on almost every Linux server, which makes it the default tool for glue work.

What you will learn

- How to create and run an executable script
- Variables, quoting and why quotes matter
- Arguments, exit codes and command substitution
- Why you should check the result of every important command

Your first script

```
#!/usr/bin/env bash
# check-disk.sh: print the usage of the root filesystem
usage=$(df --output=pcent / | tail -n 1 | tr -dc '0-9')
echo "Root filesystem is ${usage}% full"
```

The first line, the **shebang**, tells the kernel which interpreter to use. Save it, make it executable and run it:

```
$ chmod +x check-disk.sh
$ ./check-disk.sh
Root filesystem is 28% full
```

`$(command)` is **command substitution**: it runs the command and inserts its output. The sample output above will differ on your machine.

Variables and quoting

```
name="deploy user"        # no spaces around =
echo "Hello, $name"       # double quotes expand variables
echo 'Hello, $name'       # single quotes print the text literally
echo "${name}_backup"     # braces mark where the name ends
```

> **Warning:** *Always quote variable expansions:* `"$file"`*, not* `$file`*. Unquoted, a value such as* `my file.txt` *splits into two words, and* `rm $file` *would try to delete* `my` *and* `file.txt`*. Quoting prevents an entire class of bugs and accidents.*

Arguments and exit codes

Scripts receive arguments as `$1`, `$2` and so on. `$#` is the count and `"$@"` is all of them, preserving spaces.

```
#!/usr/bin/env bash
service="$1"
if [ -z "$service" ]; then
  echo "usage: $0 SERVICE" >&2     # >&2 sends the message to stderr
  exit 2
fi
systemctl is-active --quiet "$service"
echo "exit status was $?"
```

Every command ends with an **exit code**: `0` means success and anything else means failure. `$?` holds the last one. Shell logic depends on it, because `if`, `&&` and `||` all test exit codes.

```
mkdir -p /srv/backups && echo "ready"          # second command runs only if the first succeeded
systemctl is-active shop || echo "shop is down"  # second runs only if the first failed
```

Use distinct exit codes for distinct problems, such as `2` for bad usage and `1` for a failed check, so that callers like cron and CI can react.

MARGA by EknathaLabs
marga.eknathalabs.com

Reading a file line by line and testing values

```
while IFS= read -r host; do
  if ping -c1 -W2 "$host" >/dev/null 2>&1; then
    echo "$host up"
  else
    echo "$host DOWN"
  fi
done < hosts.txt
```

`IFS= read -r` reads each line unchanged, without trimming spaces or interpreting backslashes. Redirecting `>/dev/null 2>&1` discards both normal output and errors. Common tests in `[ ... ]` are `-f` (file exists), `-d` (directory exists), `-z` (string is empty), `-n` (string is not empty), `-eq` (numbers equal) and `==` (strings equal).

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Script basics fits together

Part	What it does	What goes wrong
Your first script	The first line, the shebang, tells the kernel which interpreter to use.	Spaces around = in assignments (name = value runs a command called name).
Variables and quoting	.	Unquoted variables.
Arguments and exit codes	Scripts receive arguments as $1, $2 and so on.	Ignoring exit codes, so the script reports success after a failure.
Reading a file line by line and	IFS= read -r reads each line unchanged, without trimming spaces or interpreting backslashes.	Reading a file line by line and testing values IFS= read -r reads each line unchanged, without trimming spaces or interpreting backslashes.

Flow: Your first script to Variables and quoting / Variables and quoting to Arguments and exit codes / Arguments and exit codes to Reading a file line by line and

Incident: the cleanup script that deleted too much

> **Incident:** *A cleanup script ran* `rm -rf $BACKUP_DIR/*` *nightly. After a config change* `BACKUP_DIR` *was never set. The shell expanded the line to* `rm -rf /*`.

Two habits would have prevented it. First, make the shell stop on unset variables with `set -u` (covered next lesson), or fail explicitly with a default error:

```
backup_dir="${BACKUP_DIR:?BACKUP_DIR must be set}"   # exits with a message if unset or empty
rm -rf -- "${backup_dir:?}"/*
```

`${VAR:?message}` aborts the script when the variable is missing or empty. Second, test destructive scripts by first replacing the dangerous command with `echo`, reading what *would* run, then removing the `echo`. The rule: **never let a variable you did not validate decide what gets deleted.**

Common mistakes

- Spaces around `=` in assignments (`name = value` runs a command called `name`)
- Unquoted variables
- Ignoring exit codes, so the script reports success after a failure
- Writing scripts that only work on your laptop because of hard-coded paths

Check yourself

What does the first line #!/usr/bin/env bash do?

Answer: Tells the kernel to run the file with bash

MARGA by EknathaLabs marga.eknathalabs.com

With file="my report.txt", what is the problem with rm $file?

Answer: Word splitting turns it into two arguments, "my" and "report.txt"

What exit code does a successful command return?

Answer: 0

What does ${BACKUP_DIR:?BACKUP_DIR must be set} do when the variable is empty?

Answer: Prints the message and exits the script

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 2. Control flow and functions

14 minutes. Use conditionals, loops, case statements and functions to turn one-off commands into reusable tools.

A health check that only prints "OK" is not much use. You need scripts that make decisions: retry a flaky call, branch on which environment you are in, and reuse the same logic for ten servers. Control flow and functions give your scripts that structure.

What you will learn

- `if`, `case` and the difference between `[ ]` and `[[ ]]`
- `for` and `while` loops
- Functions, local variables and return codes
- A retry helper you can reuse

Conditionals

```
if [[ "$ENVIRONMENT" == "prod" && -f /etc/shop/maintenance ]]; then
  echo "Maintenance flag is set in production"
elif [[ -z "$ENVIRONMENT" ]]; then
  echo "ENVIRONMENT is not set" >&2
else
  echo "Continuing in $ENVIRONMENT"
fi
```

`[[ ... ]]` is Bash's extended test. It supports `&&`, `||`, pattern matching (`[[ $name == web-* ]]`) and does not word-split unquoted variables, so it is safer than the older `[ ... ]`. Use `(( ... ))` for arithmetic: `if (( count > 3 )); then`.

`case` is cleaner than a chain of `elif` when you compare one value with several patterns:

```
case "$1" in
  start)   systemctl start shop ;;
  stop)    systemctl stop shop ;;
  restart) systemctl restart shop ;;
  *)       echo "usage: $0 {start|stop|restart}" >&2; exit 2 ;;
esac
```

Loops

```
for host in web01 web02 web03; do
  echo "== $host =="
  ssh "$host" uptime
done

for f in /var/log/app/*.log; do
  [[ -e "$f" ]] || continue      # skip when the glob matches nothing
  gzip -- "$f"
done

attempt=1
while (( attempt <= 5 )); do
  curl -fsS http://localhost:8080/health && break
  sleep $(( attempt * 2 ))
  (( attempt++ ))
done
```

`curl -fsS` fails on HTTP errors (`-f`), stays quiet (`-s`) but still shows errors (`-S`). That makes its exit code meaningful for scripts. In the last loop the wait grows each time, which is a simple **backoff** that gives a starting service time to become ready.

MARGA by EknathaLabs marga.eknathalabs.com

Functions

```
log() {
  printf '%s %s\n' "$(date '+%Y-%m-%dT%H:%M:%S')" "$*" >&2
}

check_port() {
  local host="$1" port="$2"          # local keeps variables inside the function
  timeout 3 bash -c "cat < /dev/null > /dev/tcp/${host}/${port}" 2>/dev/null
}

if check_port db01 5432; then
  log "database reachable"
else
  log "database unreachable"
  exit 1
fi
```

Functions **return an exit code**, not text. `return 0` is success. Use `local` for every variable inside a function, otherwise it leaks into the rest of the script. Send log messages to stderr (`>&2`) so that stdout stays clean for data that another command might consume.

A reusable retry function

```
retry() {
  local tries="$1" delay="$2"; shift 2
  local n=1
  until "$@"; do
    if (( n >= tries )); then
      log "giving up after $n attempts: $*"
      return 1
    fi
    log "attempt $n failed, retrying in ${delay}s"
    sleep "$delay"
    (( n++ ))
  done
}

retry 5 3 curl -fsS http://localhost:8080/health
```

`shift 2` drops the first two arguments so `"$@"` contains only the command to run. Because the function takes any command, you can reuse it for downloads, database checks or deploy steps.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Control flow and functions fits together

Part	What it does	What goes wrong
Conditionals	[[.]] is Bash's extended test.	Using [$var = x] with an unquoted variable that can be empty.
Loops	curl -fsS fails on HTTP errors (-f), stays quiet (-s) but still shows errors (-S).	Looping over ls output instead of a glob such as *.log.
Functions	Functions return an exit code, not text.	Forgetting local in functions.
A reusable retry function	shift 2 drops the first two arguments so "$@" contains only the command to run.	Using fixed sleep calls instead of checking readiness.

Flow: Conditionals to Loops / Loops to Functions / Functions to A reusable retry function

Incident: the deploy that "succeeded" against a dead app

> **Incident:** *A deploy script restarted the service and printed "Deploy complete" before the application finished loading. Smoke tests then failed intermittently, and one run marked the release healthy while the service was still down.*

MARGA by EknathaLabs
marga.eknathalabs.com

The script waited a fixed `sleep 5`. Startup time varies with load, so the wait was sometimes too short. Replace sleeps with a **readiness loop** that checks the thing you care about and fails clearly:

```
systemctl restart shop
if ! retry 20 3 curl -fsS http://localhost:8080/health >/dev/null; then
  log "shop did not become healthy"
  journalctl -u shop -n 20 --no-pager >&2
  exit 1
fi
log "shop is healthy"
```

Now the script only reports success when the health endpoint answers, and on failure it prints the last log lines so the person on call sees the cause immediately.

Predict the output

Common mistakes

- Using `[ $var = x ]` with an unquoted variable that can be empty
- Forgetting `local` in functions
- Looping over `ls` output instead of a glob such as `*.log`
- Using fixed `sleep` calls instead of checking readiness

Check yourself

Why prefer [[ ]] over [ ] in Bash scripts?

Answer: It avoids word splitting on unquoted variables and supports patterns and && and ||

What does local do inside a function?

Answer: Limits the variable to the function so it does not leak into the rest of the script

In the retry function, what does shift 2 accomplish?

Answer: Removes the first two arguments so "$@" holds only the command to run

Why is a readiness loop better than sleep 5 after restarting a service?

Answer: It waits only as long as needed and fails clearly if the service never becomes healthy

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 3. Text processing with grep, sed and awk

15 minutes. Slice logs and command output with pipes, grep, cut, sort, uniq, sed and awk to answer questions in one line.

A 2 GB access log contains the answer to "which clients hit us hardest at 14:03?" You do not need a dashboard or a script file. A pipeline of small tools can answer it in seconds. This is one of the highest-value skills for operations work, because most data on a server is text.

What you will learn

- How pipes connect small tools
- `grep`, `cut`, `sort`, `uniq` for counting and filtering
- `sed` for substitutions and `awk` for columns and sums
- How to build a pipeline step by step

The pipeline idea

Each Unix tool does one job and reads stdin and writes stdout. The pipe `|` sends one program's output to the next program's input.

```
$ cat access.log | head -2
203.0.113.7 - - [24/Oct/2026:14:03:11 +0000] "GET /cart HTTP/1.1" 502 157 "-" "Mozilla/5.0"
198.51.100.4 - - [24/Oct/2026:14:03:12 +0000] "GET /health HTTP/1.1" 200 2 "-" "kube-probe/1.29"
```

This is sample data in the common web server log format. Fields are separated by spaces, so field 1 is the client address, `$7` is the path and `$9` is the status code.

Filter and count

```
$ grep -c ' 502 ' access.log
41
$ grep -v 'kube-probe' access.log | wc -l
18420
$ awk '{print $1}' access.log | sort | uniq -c | sort -rn | head -3
   9402 203.0.113.7
   5120 198.51.100.22
    977 192.0.2.88
```

The last line is a classic: extract the client address, `sort` so identical values are adjacent, `uniq -c` counts runs, and `sort -rn` puts the largest first. `uniq` only collapses *adjacent* duplicates, which is why the first `sort` matters.

Tool	Use it for	Example	
`grep`	Keep or drop lines	`` `grep -E 'ERROR	WARN' app.log`
`cut`	Take fixed delimiters	`cut -d: -f1 /etc/passwd`	
`sort`, `uniq`	Order and count	`sort -u`, `uniq -c`	
`sed`	Edit text in a stream	`sed 's/old/new/g'`	
`awk`	Columns, conditions, math	`awk '$9==502 {n++} END {print n}'`	
`tr`, `wc`, `head`, `tail`	Small helpers	`wc -l`, `tail -n 100`	

sed: substitute and delete

```
sed 's/DEBUG/INFO/g' app.log              # replace in the output only
sed -i.bak 's/^port=.*/port=8081/' app.conf # edit in place, keeping app.conf.bak
sed -n '10,20p' app.log                   # print lines 10 to 20
sed '/^#/d;/^$/d' app.conf                # drop comments and blank lines
```

MARGA by EknathaLabs marga.eknathalabs.com

`-i.bak` edits the file in place and keeps a backup, which makes risky edits reversible. Without `-i`, `sed` prints the result and leaves the file alone, so test that way first.

awk: columns, conditions and sums

```
$ awk '$9 >= 500 {print $9}' access.log | sort | uniq -c
     41 502
      3 503
$ awk '{bytes += $10} END {printf "%.1f MB\n", bytes/1024/1024}' access.log
38.4 MB
$ awk -F: '$3 >= 1000 {print $1}' /etc/passwd
deploy
```

`awk` runs a rule for each line: `pattern {action}`. `END` runs after the last line, which is how you print totals. `-F:` changes the field separator to a colon.

Build pipelines step by step

Never write a long pipeline in one go. Run the first stage, look at the output, then add the next stage.

```
grep ' 502 ' access.log                          # 1. are these the lines I want?
grep ' 502 ' access.log | awk '{print $4}'       # 2. is field 4 the timestamp?
grep ' 502 ' access.log | awk '{print $4}' | cut -c2-18 | sort | uniq -c   # 3. per minute
```

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Text processing with grep, sed and awk fits together

Part	What it does	What goes wrong
The pipeline idea	Each Unix tool does one job and reads stdin and writes stdout.	Forgetting that field numbers depend on the log format.
Filter and count	The last line is a classic: extract the client address, sort so identical values are adjacent, uniq -c counts runs, and sort -rn puts the largest first.	Using uniq -c without sorting first.
sed: substitute and delete	-i.bak edits the file in place and keeps a backup, which makes risky edits reversible.	Editing a file with sed -i without a backup.
awk: columns, conditions and sums	awk runs a rule for each line: pattern {action}.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Build pipelines step by step	Never write a long pipeline in one go. Run the first stage, look at the output, then add the next stage.	Parsing structured data such as JSON with grep; use jq for that.

Flow: The pipeline idea to Filter and count / Filter and count to sed: substitute and delete / sed: substitute and delete to awk: columns, conditions and sums / awk: columns, conditions and sums to Build pipelines step by step

Incident: which endpoint is failing?

Incident: *The 502 count jumped. You need to know whether one path or the whole site is failing.*

```
$ awk '$9 == 502 {print $7}' access.log | sort | uniq -c | sort -rn | head -3
     38 /cart
      3 /checkout
```

Almost all failures hit `/cart`, so the problem is likely in the service behind that path rather than in nginx or the network. Next, check whether the failures cluster in time:

```
$ awk '$9 == 502 {print substr($4, 2, 17)}' access.log | uniq -c
     41 24/Oct/2026:14:03
```

All 41 happened in one minute, which points to a short event such as a restart, not a steady fault. In two commands you narrowed "502s" to "the cart service, one minute, 14:03", which tells you where to read the next log.

MARGA by EknathaLabs marga.eknathalabs.com

Common mistakes

- Using `uniq -c` without sorting first
- Editing a file with `sed -i` without a backup
- Parsing structured data such as JSON with `grep`; use `jq` for that
- Forgetting that field numbers depend on the log format

Check yourself

Why must you sort before uniq -c?

Answer: uniq only collapses adjacent duplicate lines

Which command prints the count of lines containing " 502 "?

Answer: grep -c ' 502 ' access.log

What does sed -i.bak 's/a/b/' file do?

Answer: Edits the file in place and saves the original as file.bak

In awk '{bytes += $10} END {print bytes}', when does the END block run?

Answer: After the last line has been processed

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 4. Advanced Bash: arguments, arrays and here-documents

15 minutes. Parse options with getopts, use arrays and associative arrays correctly, write here-documents, and combine commands with process substitution.

Once a script is more than a dozen lines, you need real argument handling, data structures and ways to generate files. Bash has all of these, with a few sharp edges you should know about.

What you will learn

- Parsing flags with `getopts` and writing a usage message
- Indexed and associative arrays, and quoting them correctly
- Here-documents for config files and messages
- Process substitution, `mapfile` and parameter expansion

Options with getopts

```
#!/usr/bin/env bash
set -euo pipefail

usage() { echo "usage: $0 [-e env] [-n] [-v] service..." >&2; exit 2; }

env="staging"; dry_run=0; verbose=0
while getopts ":e:nvh" opt; do
  case "$opt" in
    e) env="$OPTARG" ;;
    n) dry_run=1 ;;
    v) verbose=1 ;;
    h) usage ;;
    :) echo "option -$OPTARG needs a value" >&2; usage ;;
    \?) echo "unknown option -$OPTARG" >&2; usage ;;
  esac
done
shift $((OPTIND - 1))
[ "$#" -ge 1 ] || usage

for svc in "$@"; do
  echo "deploying $svc to $env (dry-run=$dry_run)"
done
```

A trailing colon after a letter in the option string means the option takes a value, and a leading colon turns on silent error handling so you print your own messages. After the loop, `shift` removes the parsed options so `"$@"` holds only the remaining arguments.

Arrays

```
hosts=(web-1 web-2 "db 1")
hosts+=(cache-1)
echo "${#hosts[@]} hosts, first is ${hosts[0]}"
for h in "${hosts[@]}"; do
  echo "checking: $h"
done

declare -A port=([web]=8080 [api]=9000)
port[metrics]=9100
for name in "${!port[@]}"; do
  echo "$name uses port ${port[$name]}"
done
```

The quoting is the whole point: `"${hosts[@]}"` expands to one word per element and preserves the element with a space in it, while an unquoted `${hosts[@]}` splits it. Associative arrays need `declare -A` and Bash 4 or newer, which excludes the default Bash on macOS.

MARGA by EknathaLabs marga.eknathalabs.com

Reading data safely

```
mapfile -t lines < <(grep -v '^#' servers.txt)
echo "read ${#lines[@]} lines"

while IFS=, read -r name ip role; do
  printf '%s has %s (%s)\n' "$name" "$ip" "$role"
done < inventory.csv

diff <(sort old.txt) <(sort new.txt)
```

`<(command)` is **process substitution**: it makes a command's output look like a file. `while read` with `IFS=` and `-r` preserves leading whitespace and backslashes. Reading with `for line in $(cat file)` splits on every space and is almost always a bug.

Here-documents

```
cat > /etc/myapp/app.conf <<EOF
listen = ${PORT:-8080}
environment = ${ENVIRONMENT:?ENVIRONMENT must be set}
EOF

cat <<'EOF'
The quotes around the delimiter stop expansion, so $HOME prints literally.
EOF
```

An unquoted delimiter allows variable expansion. Quoting the delimiter prints the text as written, which is what you want when generating scripts or Kubernetes manifests that contain dollar signs.

Parameter expansion you will use daily

```
file="/var/log/app/server.log.gz"
echo "${file##*/}"        # server.log.gz  (basename)
echo "${file%/*}"         # /var/log/app   (dirname)
echo "${file%.gz}"        # strip suffix
name="${1:-world}"        # default value
echo "${name^^}"          # uppercase
```

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Advanced Bash: arguments, arrays and... fits together

Part	What it does	What goes wrong
Options with getopts	A trailing colon after a letter in the option string means the option takes a value, and a leading colon turns on silent error handling so you print your own messages.	Forgetting shift after getopts.
Arrays	The quoting is the whole point: "${hosts[@]}" expands to one word per element and preserves the element with a space in it, while an unquoted ${hosts[@]} splits it.	Using associative arrays on systems with Bash 3.
Reading data safely	<(command) is process substitution: it makes a command's output look like a file.	Trusting echo with data that starts with a dash; prefer printf.
Here-documents	An unquoted delimiter allows variable expansion.	Unquoted $var and ${array[@]}.
Parameter expansion you will use	.	Looping over $(ls) or $(cat file).

Flow: Options with getopts to Arrays / Arrays to Reading data safely / Reading data safely to Here-documents / Here-documents to Parameter expansion you will use

MARGA by EknathaLabs marga.eknathalabs.com

Incident: the filename with a space

> **Incident:** *A cleanup script loops* `for f in $(ls /data/uploads)` *and runs* `rm $f`*. A customer uploads a file named* `report 2024.pdf`*. The loop deletes* `report` *and* `2024.pdf` *as two separate names, and in another run an empty variable turns* `rm $f` *into a command with no operand.*

Word splitting is the cause. Use globs, quote every expansion and guard with a test:

```
for f in /data/uploads/*; do
  [ -e "$f" ] || continue
  if [ "$(find "$f" -mtime +30 | wc -l)" -gt 0 ]; then
    echo "removing: $f"
    rm -- "$f"
  fi
done
```

`rm --` stops option parsing so a file named `-rf` is treated as a name. Run `shellcheck script.sh` as part of your pipeline; it catches unquoted variables like this automatically.

Common mistakes

- Looping over `$(ls)` or `$(cat file)`
- Unquoted `$var` and `${array[@]}`
- Using associative arrays on systems with Bash 3
- Forgetting `shift` after getopts
- Trusting `echo` with data that starts with a dash; prefer `printf`

Check yourself

What does "${hosts[@]}" do when quoted?

Answer: Expands each element as a separate word, preserving spaces inside elements

What does a leading colon in a getopts option string do?

Answer: Enables silent mode so you handle errors yourself

What is the difference between an unquoted and a quoted here-document delimiter?

Answer: The unquoted form expands variables; the quoted form is literal

Why is for f in $(ls) a bad pattern?

Answer: Output is split on whitespace, so names with spaces break

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 5. Working with APIs and JSON using curl and jq

14 minutes. Call REST APIs from the shell with curl, process JSON with jq, handle errors and pagination, and keep secrets out of command history.

Modern infrastructure is controlled through APIs: cloud providers, Kubernetes, monitoring systems and chat tools. `curl` talks to them, and `jq` turns their JSON responses into exactly the fields you need.

What you will learn

- The curl flags you need for scripts
- Selecting, filtering and reshaping JSON with jq
- Checking HTTP status codes and retrying
- Passing credentials safely

curl for scripts

```
$ curl -sS https://api.example.com/v1/health
$ curl -sS -o /dev/null -w "%{http_code} %{time_total}s\n" https://api.example.com/v1/health
$ curl -fsS -m 10 --retry 3 --retry-delay 2 https://api.example.com/v1/servers
$ curl -fsS -X POST https://api.example.com/v1/servers -H "Content-Type: application/json" -d '{"n \
    ame":"web-3","size":"small"}'
```

The flags to remember: `-s` silences the progress meter, `-S` still shows errors, `-f` makes curl exit non-zero for HTTP 400 and above, `-m` sets a time limit, and `-w` prints details such as the status code. Without `-f`, a 500 response with an HTML error page looks like success to your script.

Credentials

Never write tokens on the command line, because they appear in the process list and shell history. Read them from the environment, a file or a secret manager.

```
#!/usr/bin/env bash
set -euo pipefail
: "${API_TOKEN:?set API_TOKEN}"

curl -fsS -m 10 \
  -H "Authorization: Bearer ${API_TOKEN}" \
  https://api.example.com/v1/servers
```

With `--config` curl can also read headers from a file or standard input, which keeps them out of `ps` output.

jq basics

```
$ echo '{"items":[{"name":"web-1","cpu":0.91,"tags":["prod"]},{"name":"db-1","cpu":0.40,"tags":["p \
    rod","db"]}]}' > servers.json
$ jq '.items[].name' servers.json
$ jq -r '.items[] | select(.cpu > 0.8) | .name' servers.json
$ jq -r '.items[] | [.name, .cpu] | @tsv' servers.json
$ jq '.items | map({host: .name, hot: (.cpu > 0.8)})' servers.json
$ jq '[.items[] | select(.tags | index("db"))] | length' servers.json
```

`-r` outputs raw strings without quotes, which is what shell variables want. `select()` filters, `map()` transforms every element, and `@tsv` or `@csv` format rows for reports. Use `jq -e` in conditions so a false or null result gives a non-zero exit code.

Building JSON safely

Never assemble JSON by gluing strings together: a quote in a value breaks the document.

```
$ jq -n --arg name "web-3" --argjson replicas 3 '{name: $name, replicas: $replicas}'
$ jq --arg v "1.8.2" '.version = $v' config.json > config.new && mv config.new config.json
```

MARGA by EknathaLabs marga.eknathalabs.com

Pagination and status checks

```
#!/usr/bin/env bash
set -euo pipefail
url="https://api.example.com/v1/servers?limit=100"
while [ -n "$url" ]; do
  resp=$(curl -fsS -m 10 -H "Authorization: Bearer ${API_TOKEN}" "$url")
  echo "$resp" | jq -r '.items[].name'
  url=$(echo "$resp" | jq -r '.next // empty')
done
```

The `// empty` fallback prevents the literal string `null` ending up in the variable when there is no next page, so the loop ends cleanly.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Working with APIs and JSON using curl and jq fits together

Part	What it does	What goes wrong
curl for scripts	The flags to remember: -s silences the progress meter, -S still shows errors, -f makes curl exit non-zero for HTTP 400 and above, -m sets a time limit, and -w prints...	Using curl without -f, so errors look like success.
Credentials	Never write tokens on the command line, because they appear in the process list and shell history.	Putting tokens on the command line.
jq basics	-r outputs raw strings without quotes, which is what shell variables want.	Forgetting -r, leaving quotes in shell variables.
Building JSON safely	Never assemble JSON by gluing strings together: a quote in a value breaks the document.	Building JSON bodies with string concatenation.
Pagination and status checks	The // empty fallback prevents the literal string null ending up in the variable when there is no next page, so the loop ends cleanly.	Ignoring pagination and seeing only the first page.

Flow: curl for scripts to Credentials / Credentials to jq basics / jq basics to Building JSON safely / Building JSON safely to Pagination and status checks

Incident: the pipeline that deployed on a 502

> **Incident:** *A deploy script runs* `curl https://api/deploy/status | jq -r .state`*. The API returns a 502 HTML page, jq fails to parse it, and because the pipeline lacks* `pipefail` *the step still exits 0. The pipeline then proceeds as if the deployment had succeeded.*

Two defects: curl did not fail on HTTP errors, and the pipeline hid the failure. The robust version:

```
set -euo pipefail
state=$(curl -fsS -m 15 --retry 3 "https://api.example.com/deploy/status" | jq -er '.state')
[ "$state" = "ready" ] || { echo "deployment is $state" >&2; exit 1; }
```

`-f` fails on 4xx and 5xx, `pipefail` propagates the error from the first command in the pipe, and `jq -e` fails when the field is missing.

Common mistakes

- Using curl without `-f`, so errors look like success
- Putting tokens on the command line
- Building JSON bodies with string concatenation
- Forgetting `-r`, leaving quotes in shell variables
- Ignoring pagination and seeing only the first page

MARGA by EknathaLabs marga.eknathalabs.com

Check yourself

What does curl -f do?

Answer: Fails with a non-zero exit code on HTTP errors such as 404 and 500

What does jq -r do?

Answer: Prints raw strings without JSON quotes

Why avoid putting an API token directly in the curl command line?

Answer: It is visible in process lists and shell history

How should you build a JSON request body in a script?

Answer: Use jq -n with --arg or --argjson so values are escaped correctly

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 6. Safe automation: strict mode, traps and scheduling

16 minutes. Make scripts fail loudly with strict mode, clean up with traps, avoid overlapping runs, and schedule them with cron or systemd timers.

A backup script has run every night for a year. One night the disk fills, the dump fails halfway, and the script keeps going. It compresses a truncated file, reports success and deletes yesterday's good backup. Automation that fails silently is worse than no automation. This lesson covers the habits that make scripts safe to run unattended.

What you will learn

- Strict mode: `set -euo pipefail`
- Cleaning up with `trap`
- Preventing two copies from running at once
- Scheduling with cron and systemd timers

Strict mode

```
#!/usr/bin/env bash
set -euo pipefail
IFS=$'\n\t'
```

Option	What it does
`-e`	Exit when a command fails
`-u`	Treat unset variables as errors
`-o pipefail`	A pipeline fails if *any* stage fails, not just the last

Without `pipefail`, `pg_dump mydb | gzip > backup.gz` reports success even when `pg_dump` fails, because `gzip` succeeds. With it, the script stops.

> **Note:** `set -e` *has exceptions: it does not trigger inside an* `if` *condition or on the left of* `&&` *and* `||`*. Strict mode is a safety net, not a replacement for checking important results yourself.*

Clean up with trap

A `trap` runs a command when the script exits or receives a signal. Use it to remove temporary files whatever happens.

```
tmp="$(mktemp -d)"
cleanup() { rm -rf -- "$tmp"; }
trap cleanup EXIT                # runs on normal exit, on error, and on Ctrl+C

pg_dump "$DB_NAME" | gzip > "$tmp/db.sql.gz"
mv "$tmp/db.sql.gz" "/backups/db-$(date +%F).sql.gz"
```

Write to a temporary location and **move into place at the end**. A move on the same filesystem is atomic, so readers never see a half-written file.

One run at a time

If a job takes longer than its schedule interval, two copies run together and can corrupt data. `flock` takes a lock and refuses to start a second copy.

```
exec 9>/var/lock/nightly-backup.lock
flock -n 9 || { echo "another backup is running" >&2; exit 0; }
```

The lock is released automatically when the script exits, even after a crash.

Scheduling

cron is the classic scheduler. Edit with `crontab -e`.

MARGA by EknathaLabs marga.eknathalabs.com

```
# minute hour day-of-month month day-of-week  command
15 2 * * *  /usr/local/bin/nightly-backup.sh >> /var/log/nightly-backup.log 2>&1
```

Cron runs with a minimal environment, so use full paths and set what the script needs. Capture output, because cron mails or drops it otherwise.

systemd timers do the same job with better logging and dependencies. A timer triggers a service unit.

```
# /etc/systemd/system/backup.timer
[Unit]
Description=Nightly backup

[Timer]
OnCalendar=*-*-* 02:15:00
Persistent=true
RandomizedDelaySec=300

[Install]
WantedBy=timers.target
```

`Persistent=true` runs a missed job after the machine was off. Enable with `systemctl enable --now backup.timer`, list with `systemctl list-timers`, and read output with `journalctl -u backup.service`.

Incident: the backup that deleted the last good copy

> **Incident:** *The nightly job ran* `pg_dump | gzip > latest.sql.gz`*, then* `find /backups -mtime +7 -delete`*. The database host was out of space,* `pg_dump` *failed, and* `latest.sql.gz` *contained a few kilobytes. Cleanup then removed older files. After a week there was no usable backup.*

A safer version applies every habit from this lesson:

```
#!/usr/bin/env bash
set -euo pipefail

backup_dir="${BACKUP_DIR:?BACKUP_DIR must be set}"
tmp="$(mktemp -d)"; trap 'rm -rf -- "$tmp"' EXIT
exec 9>/var/lock/nightly-backup.lock; flock -n 9 || exit 0

out="$backup_dir/db-$(date +%F).sql.gz"
pg_dump "$DB_NAME" | gzip > "$tmp/dump.gz"

# validate before replacing anything
gzip -t "$tmp/dump.gz"
[[ "$(stat -c %s "$tmp/dump.gz")" -gt 1048576 ]] || { echo "dump suspiciously small" >&2; exit 1;  \
    }

mv "$tmp/dump.gz" "$out"
find "$backup_dir" -name 'db-*.sql.gz' -mtime +7 -delete   # only after a verified new backup
```

If `pg_dump` fails, `pipefail` stops the script before any deletion. The size and integrity checks catch truncated output. Old backups are removed only after a new one has been verified. Finally, a **restore test** matters more than any check: schedule a monthly job that actually restores the latest backup into a scratch database.

Diagram: Where a safe script protects you

Part	What it does	What goes wrong
Start	The script begins with set -euo pipefail and a trap for cleanup.	Without it, an error in the middle is ignored and the script goes on.
Validate input	Check that required variables are set and paths are not empty.	An empty variable in rm -rf $DIR/ can become rm -rf /.
Dry run	Print the actions first with a --dry-run flag.	Skipping it means the first real run is also the first test.
Do the work	Use quoted variables, a lock file and clear logging.	Two copies running at once can corrupt data without a lock.
Cleanup and exit code	The trap removes temp files and returns a meaningful code.	If the exit code is always 0, the scheduler thinks a failure succeeded.

MARGA by EknathaLabs marga.eknathalabs.com

Flow: Start to Validate input / Start to Dry run / Validate input to Do the work / Dry run to Do the work / Do the work to Cleanup and exit code

Common mistakes

- Cleaning up old data before confirming the new data is good
- Relying on `set -e` and never checking critical results
- Cron jobs that depend on `PATH` or variables from your interactive shell
- Never testing a restore

Check yourself

Why does set -o pipefail matter for pg_dump | gzip > file?

Answer: Without it, the pipeline succeeds when gzip succeeds even if pg_dump failed

What does trap cleanup EXIT do?

Answer: Runs cleanup when the script exits, whether normally, by error or by interrupt

Why write to a temp file and mv it into place?

Answer: A rename on the same filesystem is atomic, so readers never see a partial file

What is the best proof that a backup works?

Answer: Regularly restoring it into a test environment and checking the data

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 7. Arrays, associative arrays and parameter expansion

13 minutes. Use Bash arrays and associative arrays for lists and lookups, and use parameter expansion to trim, replace and default values without calling external programs.

Many shell scripts split strings with `cut` and `awk` or loop over words in a variable, and then break when a value contains a space. Bash has real arrays and a rich set of expansions that handle these jobs safely and without starting extra processes. They make scripts shorter, faster and less fragile.

What you will learn

- Indexed and associative arrays
- Looping over arrays safely
- Parameter expansion: defaults, trimming, replacing, lengths and slices
- Reading command output into arrays

Indexed arrays

```
hosts=(web1 web2 "db 1")
hosts+=(cache1)
echo "${#hosts[@]}"           # count: 4
echo "${hosts[2]}"            # db 1
for h in "${hosts[@]}"; do    # quote and use [@] to keep elements intact
  echo "checking $h"
done
echo "${hosts[@]:1:2}"        # slice: two elements starting at index 1
```

The form `"${hosts[@]}"` expands to one word per element, even for elements with spaces. The unquoted form splits on spaces and breaks the list.

Reading output into an array

```
mapfile -t files < <(find /var/log -name '*.log' -mtime -1)
printf 'found %d files\n' "${#files[@]}"
```

`mapfile` (also `readarray`) reads one line per element and keeps spaces. This is safer than `files=($(find ...))`, which splits on any whitespace.

Associative arrays

```
declare -A owner
owner[web1]=alice
owner[db1]=bob
for k in "${!owner[@]}"; do echo "$k is owned by ${owner[$k]}"; done
[[ -v owner[web2] ]] || echo "web2 has no owner"
```

Use these for lookups such as host to owner or service to port. They need Bash 4 or later, which macOS does not ship by default.

MARGA by EknathaLabs
marga.eknathalabs.com

Parameter expansion

Form	Result
`${name:-default}`	`default` if name is unset or empty
`${name:?message}`	Stop with a message if unset or empty
`${#name}`	Length of the value
`${name%.log}`	Remove the shortest match from the end
`${name##*/}`	Remove the longest match from the start (the file name)
`${name/old/new}`	Replace the first match
`${name//old/new}`	Replace all matches
`${name^^}` / `${name,,}`	Upper or lower case

```
path=/var/log/app/error.log
echo "${path##*/}"      # error.log
echo "${path%/*}"       # /var/log/app
echo "${path%.log}.old" # /var/log/app/error.old
: "${DB_HOST:?DB_HOST must be set}"
```

Where to stop

When a script needs nested data or heavy parsing, use `jq` or move to Python. Arrays are for lists and simple lookups.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Arrays, associative arrays and parameter expansion fits together

Part	What it does	What goes wrong
Indexed arrays	The form "${hosts[@]}" expands to one word per element, even for elements with spaces.	Unquoted ${array[@]}.
Reading output into an array	mapfile (also readarray) reads one line per element and keeps spaces.	Parsing ls output.
Associative arrays	Use these for lookups such as host to owner or service to port.	Using associative arrays on shells that lack them.
Parameter expansion	.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Where to stop	When a script needs nested data or heavy parsing, use jq or move to Python.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Indexed arrays to Reading output into an array / Reading output into an array to Associative arrays / Associative arrays to Parameter expansion / Parameter expansion to Where to stop

Incident: the backup that skipped half the files

Incident: *A backup script reports success but misses files whose names contain spaces.*

```
$ files=($(ls /srv/reports)); echo "${#files[@]}"
212
$ ls /srv/reports | wc -l
140
```

The script built its list by word-splitting the output of `ls`, so each name with a space became several entries that did not exist. The fix used `mapfile -t files < <(find ... -print0)` style reading with a quoted loop, and the script compared the count of files copied with the count found.

Common mistakes

- Parsing `ls` output

MARGA by EknathaLabs
marga.eknathalabs.com

- Unquoted `${array[@]}`
- Using associative arrays on shells that lack them
- Forgetting `:?` checks for required variables

Check yourself

Which form expands an array to one word per element, keeping spaces inside elements?

Answer: "${arr[@]}"

What does ${path##*/} return for /var/log/app.log?

Answer: app.log

What does ${DB_HOST:?must be set} do when DB_HOST is empty?

Answer: Stops the script with the message

Why is mapfile safer than files=($(command))?

Answer: It reads one line per element without splitting on spaces or expanding wildcards

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 8. Building and testing shell tools

13 minutes. Turn a script into a small tool with options, help text, exit codes and tests using getopts, shellcheck and bats, and run them automatically.

A script that only its author understands becomes a risk the day that person is away. A small tool has a clear interface: options, a help message, meaningful exit codes, and tests. Bash is not the best language for large programs, but a few good habits make shell tools reliable enough to share with a team.

What you will learn

- Parsing options with `getopts`
- A standard script layout with functions and a `main`
- Exit codes and error messages
- Testing with `bats` and checking with `shellcheck`
- Making scripts safe to source and to run twice

Layout

```
#!/usr/bin/env bash
set -Eeuo pipefail

usage() { cat <<'EOF'
Usage: rotate.sh [-d DIR] [-k DAYS] [-n]
  -d DIR   directory to clean (required)
  -k DAYS  keep files newer than DAYS (default 14)
  -n       dry run, only print what would be removed
EOF
}

main() {
  local dir="" keep=14 dry=0 opt
  while getopts ":d:k:nh" opt; do
    case $opt in
      d) dir=$OPTARG ;;
      k) keep=$OPTARG ;;
      n) dry=1 ;;
      h) usage; return 0 ;;
      :) echo "option -$OPTARG needs a value" >&2; return 64 ;;
      *) usage >&2; return 64 ;;
    esac
  done
  [[ -n $dir && -d $dir ]] || { echo "error: -d must be an existing directory" >&2; return 64; }
  [[ $keep =~ ^[0-9]+$ ]] || { echo "error: -k must be a number" >&2; return 64; }
  if (( dry )); then
    find "$dir" -type f -mtime "+$keep" -print
  else
    find "$dir" -type f -mtime "+$keep" -delete
  fi
}

if [[ ${BASH_SOURCE[0]} == "$0" ]]; then main "$@"; fi
```

Putting logic in functions and calling `main` only when the file is run (not sourced) lets tests load the functions without running the tool.

Exit codes and messages

Use 0 for success, 1 for a general failure and 64 for a usage error (the convention from `sysexits`). Send errors to standard error (`>&2`) and normal output to standard output, so pipelines keep working. Make the message say what was wrong and what the user can do.

MARGA by EknathaLabs marga.eknathalabs.com

Tests with bats

```
#!/usr/bin/env bats

setup() { TMP=$(mktemp -d); touch -d '30 days ago' "$TMP/old.log"; touch "$TMP/new.log"; }
teardown() { rm -rf "$TMP"; }

@test "dry run lists old files and keeps them" {
  run ./rotate.sh -d "$TMP" -k 14 -n
  [ "$status" -eq 0 ]
  [[ "$output" == *old.log* ]]
  [ -e "$TMP/old.log" ]
}

@test "missing directory is a usage error" {
  run ./rotate.sh -d /nope
  [ "$status" -eq 64 ]
}
```

Run `shellcheck rotate.sh` and `bats tests/` in your pipeline. Use a temporary directory for test data and never point tests at real paths.

Idempotence and safety

A tool you may run twice should give the same result the second time. Prefer `mkdir -p`, `ln -sfn`, and checks such as "does it exist" before changes. Provide a dry-run option for anything destructive, and print what you are about to do.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Building and testing shell tools fits together

Part	What it does	What goes wrong
Layout	Putting logic in functions and calling main only when the file is run (not sourced) lets tests load the functions without running the tool.	No validation of required options.
Exit codes and messages	Use 0 for success, 1 for a general failure and 64 for a usage error (the convention from sysexits).	Mixing messages and results on standard output.
Tests with bats	Run shellcheck rotate.sh and bats tests/ in your pipeline.	Tests that touch real directories.
Idempotence and safety	A tool you may run twice should give the same result the second time.	No dry-run mode for destructive tools.

Flow: Layout to Exit codes and messages / Exit codes and messages to Tests with bats / Tests with bats to Idempotence and safety

Incident: the cleanup that ran with the wrong variable

> **Incident:** *A cleanup script deletes the wrong directory because a path variable was empty after a typo in an option name.*

```
$ ./clean.sh --dri /data/tmp
removing files under /
```

The tool ignored unknown options and fell back to an empty directory value. The rewrite validated every option, required `-d` to be an existing directory, refused to run on `/`, offered a dry run, and had tests for each of these cases.

Common mistakes

- No validation of required options
- Mixing messages and results on standard output

- Tests that touch real directories
- No dry-run mode for destructive tools

Check yourself

Which builtin helps parse options such as -d dir and -n?

Answer: getopts

Where should error messages go?

Answer: Standard error

Why call main only if the script is not sourced?

Answer: Tests can source the file and call its functions without running the tool

What does a dry-run option provide?

Answer: A preview of what the tool would do, without changing anything

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 9. Temporary files, locks and atomic writes

12 minutes. Create temporary files safely, replace files without leaving a half-written copy, and stop two copies of a script running at once.

A script writes a report straight into the file that a web page serves, and a visitor loads the page halfway through. Another script uses /tmp/work as a scratch file, and two runs overwrite each other. These are the everyday ways shell scripts corrupt data, and each has a small, standard cure.

What you will learn

- Create unique temporary files and directories with mktemp
- Clean up with trap, even on failure
- Write a file atomically: write elsewhere, then rename
- Prevent overlapping runs with flock

Never invent a temporary name

A fixed name such as `/tmp/work` is shared by every run, and another user can create it first and trick your script. `mktemp` creates a file or directory with a random name and safe permissions.

```
tmp=$(mktemp)            # a new empty file, mode 600
work=$(mktemp -d)        # a new private directory
echo "scratch" > "$tmp"
```

Store the name in a variable and quote it every time you use it. Prefer a directory when a script needs several files, because one `rm -r` cleans them all.

Clean up with trap

Temporary files must disappear even if the script fails or you press Ctrl+C. A `trap` on `EXIT` runs a command whenever the script ends, for any reason except a hard kill.

```
#!/bin/bash
set -euo pipefail
work=$(mktemp -d)
trap 'rm -rf "$work"' EXIT

curl -fsS https://example.com/data.json -o "$work/data.json"
jq .items "$work/data.json" > "$work/items.json"
```

Set the trap straight after creating the directory so that there is no gap in which a failure leaves it behind. Single quotes make the variable expand when the trap runs, not when it is set.

Write atomically

Writing straight into the final file means readers can see it half finished, and a crash leaves a broken file. The pattern is: write a complete new file in the same directory, then rename it over the old one. A rename within one filesystem is atomic, so readers see either the old file or the new one, never a mix.

```
out=/srv/www/report.html
new=$(mktemp "$out.XXXXXX")        # same directory, same filesystem
generate_report > "$new"
chmod 644 "$new"
mv -f "$new" "$out"
```

The temporary file must be in the same directory (or at least the same filesystem). Across filesystems `mv` copies and deletes, which is not atomic.

One copy at a time with flock

If a job runs every minute and sometimes takes two, you get two copies working on the same data. `flock` takes a lock on a file and releases it automatically when the process ends, even if it crashes.

MARGA by EknathaLabs
marga.eknathalabs.com

```
#!/bin/bash
exec 9>/var/lock/report.lock
if ! flock -n 9; then
  echo "report already running" >&2
  exit 0
fi
# ... the real work runs while the lock is held ...
```

The `-n` option fails at once instead of waiting. A hand-made lock file (`touch lock; ... rm lock`) is worse, because a crash leaves it behind and the job never runs again.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Temporary files, locks and atomic writes fits together

Part	What it does	What goes wrong
Never invent a temporary name	A fixed name such as /tmp/work is shared by every run, and another user can create it first and trick your script.	Creating the temporary file and the trap in the wrong order.
Clean up with trap	Temporary files must disappear even if the script fails or you press Ctrl+C.	Using a plain lock file that a crash leaves behind.
Write atomically	Writing straight into the final file means readers can see it half finished, and a crash leaves a broken file.	Renaming across filesystems and expecting it to be atomic.
One copy at a time with flock	If a job runs every minute and sometimes takes two, you get two copies working on the same data.	The -n option fails at once instead of waiting.

Flow: Never invent a temporary name to Clean up with trap / Clean up with trap to Write atomically / Write atomically to One copy at a time with flock

Incident: Half a report

> **Incident:** *A nightly job regenerates the status page in place. At 02:00 customers sometimes saw a page that stopped in the middle of a table.*

```
$ ls -l /srv/www/status.html
-rw-r--r-- 1 www www 18432 Jan 6 02:00 status.html   # normally 52 KB
$ grep -n "status.html" /opt/jobs/status.sh
12: render_status > /srv/www/status.html
```

The script truncated the live file and then wrote it over several seconds. Visitors in that window got a partial page. The fix was to render into a temporary file in the same directory, check it was not empty, and rename it over the old file, and to add `flock` so that a slow run could not overlap the next one.

Common mistakes

- Using fixed names in /tmp
- Creating the temporary file and the trap in the wrong order
- Renaming across filesystems and expecting it to be atomic
- Using a plain lock file that a crash leaves behind

Check yourself

Why use mktemp instead of /tmp/work?

Answer: It makes a unique file with safe permissions, so runs do not collide and nobody can pre-create it

What makes a file replacement atomic?

Answer: Writing a new file in the same filesystem and renaming it over the old one

MARGA by EknathaLabs marga.eknathalabs.com

Which trap runs on every normal or failing exit?

Answer: trap CMD EXIT

What is the advantage of flock over a plain lock file?

Answer: The lock is released automatically when the process ends, even after a crash

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 10. Portable scripts, linting and style

11 minutes. Write scripts that run on different systems, catch mistakes with ShellCheck, and keep them readable for the next person.

A script that worked on your laptop fails on the build server with "bad substitution", and nobody can tell why. The server runs `sh`, which is not Bash. A little discipline about the shebang, the features you use, and an automatic linter prevents most of these surprises.

What you will learn

- The difference between sh and bash and how to choose the shebang
- Common Bash-only features that break under sh
- Using ShellCheck to find bugs before they run
- Readable style: functions, names, comments and usage text

sh is not bash

`#!/bin/sh` asks for a minimal POSIX shell. On Debian and Ubuntu that is `dash`, which is fast but lacks many Bash features. `#!/bin/bash` asks for Bash. Pick the one you really need, and write it on the first line.

```
#!/usr/bin/env bash
set -euo pipefail
```

`env bash` finds Bash on the path, which helps on systems that keep it elsewhere. Features that only Bash has include arrays, `[[ ... ]]`, `${var,,}`, process substitution `<(cmd)`, `pipefail` and `source`. If a script must run under plain `sh`, avoid all of them and test it with `dash script.sh`.

Find the mistakes with ShellCheck

ShellCheck reads a script and reports problems that are easy to miss: unquoted variables, wrong tests, useless uses of cat, and Bash-only features under an `sh` shebang. Each message has a code such as SC2086 that you can look up.

```
$ shellcheck deploy.sh
In deploy.sh line 7:
rm -rf $TARGET/*
       ^-- SC2086: Double quote to prevent globbing and word splitting.
```

Run it in your editor and in the pipeline, and treat warnings as bugs. When you disagree, silence one line with a comment such as `# shellcheck disable=SC2034` and explain why.

Readable by the next person

Scripts live longer than expected. A few habits make them easy to change at 03:00:

```
#!/usr/bin/env bash
# Rotate old reports. Usage: rotate.sh DIR DAYS
set -euo pipefail

usage() { echo "usage: $0 DIR DAYS" >&2; exit 2; }
[[ $# -eq 2 ]] || usage

readonly dir=$1 days=$2

main() {
  find "$dir" -type f -mtime +"$days" -print -delete
}
main
```

Put work in functions, name variables for what they hold, quote every expansion, and print a usage line for wrong input. Use `readonly` for values that must not change. Keep a short comment at the top that says what the script does and how to call it.

MARGA by EknathaLabs
marga.eknathalabs.com

Test the edges

Run the script on the situations that break it: an empty folder, a name with a space, no arguments, and a missing command. The `bats` test framework lets you write these checks as files, but even a few manual runs after `set -x` find most problems.

```
$ bash -n script.sh        # syntax check only
$ bash -x script.sh arg    # trace every command
$ shellcheck -x script.sh  # follow sourced files
```

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Portable scripts, linting and style fits together

Part	What it does	What goes wrong
sh is not bash	#!/bin/sh asks for a minimal POSIX shell.	Using #!/bin/sh and then Bash-only features.
Find the mistakes with ShellCheck	ShellCheck reads a script and reports problems that are easy to miss: unquoted variables, wrong tests, useless uses of cat, and Bash-only features under an sh shebang.	Running scripts with sh script.sh and bypassing the shebang.
Readable by the next person	Scripts live longer than expected.	Ignoring ShellCheck warnings because the script works today.
Test the edges	Run the script on the situations that break it: an empty folder, a name with a space, no arguments, and a missing command.	Leaving out usage text so nobody knows the arguments.

Flow: sh is not bash to Find the mistakes with ShellCheck / Find the mistakes with ShellCheck to Readable by the next person / Readable by the next person to Test the edges

Incident: Works on my laptop

> **Incident:** *A pipeline step that ran a deployment script failed only on the build machine with a syntax error on a line that looked fine.*

```
$ head -1 deploy.sh
#!/bin/sh
$ ls -l /bin/sh
lrwxrwxrwx 1 root root 4 /bin/sh -> dash
$ sh deploy.sh
deploy.sh: 14: Syntax error: "(" unexpected
```

The script used a Bash array. On the laptop, /bin/sh was Bash, so it worked; on the build server it is dash. The team changed the shebang to bash, added ShellCheck to the pipeline so this class of error fails the build, and documented that scripts use Bash 4 or newer.

Common mistakes

- Using #!/bin/sh and then Bash-only features
- Running scripts with sh script.sh and bypassing the shebang
- Ignoring ShellCheck warnings because the script works today
- Leaving out usage text so nobody knows the arguments

Check yourself

What does #!/bin/sh promise?

Answer: Only POSIX shell features; it may be a minimal shell such as dash

Which tool finds unquoted variables and other likely bugs before you run a script?

Answer: ShellCheck

MARGA by EknathaLabs
marga.eknathalabs.com

Which of these is a Bash-only feature?

Answer: Arrays

What does readonly do?

Answer: Stops a variable from being changed later in the script

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 11. Debugging shell scripts

14 minutes. Trace a script with set -x, catch silent failures with strict mode and shellcheck, and find why a script works in your terminal but fails under cron or CI.

A shell script that works on your laptop and fails at 03:00 in cron is one of the most common operations problems. The cause is rarely the logic. It is usually the environment, the quoting, or an error that nothing noticed. This lesson gives you tools that make a script show its work.

What you will learn

- Tracing with `set -x` and a useful `PS4`
- Strict mode and what it does not catch
- Finding mistakes before running with `shellcheck` and `bash -n`
- Why scripts differ between a terminal and cron or CI

See every command: tracing

```
bash -x ./backup.sh                # trace one run
set -x                             # or turn tracing on inside the script
export PS4='+ ${BASH_SOURCE##*/}:${LINENO}: '   # show file and line for each traced command
```

Each traced line starts with the `PS4` prefix, so you see which line ran and with which expanded values. Turn tracing on only around the suspect part (`set -x` ... `set +x`) to keep the output short, and remember it may print secrets.

Fail loudly

```
#!/usr/bin/env bash
set -Eeuo pipefail
trap 'echo "failed at line $LINENO: $BASH_COMMAND" >&2' ERR
```

`-e` exits on an error, `-u` treats unset variables as errors, `pipefail` makes a pipeline fail when any part fails, and the `ERR` trap tells you where. Strict mode is not magic: commands in an `if` test, in `&&` chains, or on the left of `||` are exempt from `-e`, so keep logic simple and check important results yourself.

Check before you run

```
$ bash -n backup.sh              # syntax check only, nothing runs
$ shellcheck backup.sh
In backup.sh line 7:
rm -rf $DIR/*
       ^--^ SC2086: Double quote to prevent globbing and word splitting.
```

`shellcheck` finds quoting mistakes, unused variables and unsafe patterns. Run it in your pipeline so every change is checked.

Terminal versus cron

Cron runs scripts with a minimal environment: a short `PATH`, no login files, a different working directory and no terminal.

Difference	Symptom	Fix
PATH is short	`command not found`	Use full paths or set PATH at the top
Working directory differs	Relative paths fail	`cd "$(dirname "$0")"` or use absolute paths
No profile loaded	Missing variables	Set variables in the script or an env file
Output is mailed or lost	You never see errors	Redirect to a log: `>> /var/log/backup.log 2>&1`

MARGA by EknathaLabs marga.eknathalabs.com

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Debugging shell scripts fits together

Part	What it does	What goes wrong
See every command: tracing	Each traced line starts with the PS4 prefix, so you see which line ran and with which expanded values.	Unquoted variables, which split on spaces and expand wildcards.
Fail loudly	-e exits on an error, -u treats unset variables as errors, pipefail makes a pipeline fail when any part fails, and the ERR trap tells you where.	Fail loudly -e exits on an error, -u treats unset variables as errors, pipefail makes a pipeline fail when any part fails, and the ERR trap...
Check before you run	shellcheck finds quoting mistakes, unused variables and unsafe patterns.	Assuming set -e catches every failure.
Terminal versus cron	Cron runs scripts with a minimal environment: a short PATH, no login files, a different working directory and no terminal.	Sending cron output nowhere.

Flow: See every command: tracing to Fail loudly / Fail loudly to Check before you run / Check before you run to Terminal versus cron

Incident: works by hand, fails at night

Incident: *A nightly backup script runs fine when started manually, but the cron job produces no archive and no error.*

```
$ crontab -l
0 2 * * * /opt/scripts/backup.sh
$ env -i /bin/sh -c '/opt/scripts/backup.sh'
/opt/scripts/backup.sh: line 9: aws: command not found
```

Running the script with a clean environment (`env -i`) reproduced the failure: `aws` lived in `/usr/local/bin`, which cron's `PATH` does not include. The fix was a full path to the command, a `PATH` line at the top, and a log redirect, so the next failure leaves evidence.

Common mistakes

- Unquoted variables, which split on spaces and expand wildcards
- Assuming `set -e` catches every failure
- Debugging only in an interactive shell
- Sending cron output nowhere

Check yourself

What does set -x do?

Answer: It prints each command, after expansion, before running it

Why does a script that works in your terminal often fail in cron?

Answer: Cron uses a minimal environment, with a short PATH and a different working directory

What does pipefail change?

Answer: A pipeline returns a failure if any command in it fails, not only the last

Which tool finds quoting mistakes without running the script?

Answer: shellcheck

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 12. Log analysis one-liners and pipelines

12 minutes. Answer real operational questions from log files with grep, awk, sort, uniq and jq, and build small reusable pipelines.

Dashboards answer questions someone planned for. Logs answer the questions nobody planned. Knowing a few shell patterns lets you explore any log quickly, even on a server with no extra tools.

What you will learn

- Filtering and extracting fields
- Counting and ranking with sort and uniq
- Time-based and percentile analysis
- JSON logs with jq and streaming tails

Count and rank

```
# top client IPs in an nginx access log
awk '{print $1}' access.log | sort | uniq -c | sort -nr | head

# status code distribution
awk '{print $9}' access.log | sort | uniq -c | sort -nr

# top failing URLs
awk '$9 >= 500 {print $7}' access.log | sort | uniq -c | sort -nr | head
```

The pattern is **extract, sort, count, rank**. `uniq -c` only counts adjacent duplicates, so always sort first.

Time windows and latency

```
# requests per minute
awk '{print substr($4, 2, 17)}' access.log | sort | uniq -c | tail

# 95th percentile of request time (last field)
awk '{print $NF}' access.log | sort -n | awk '{a[NR]=$1} END {print a[int(NR*0.95)]}'

# errors in the last 10 minutes of a systemd service
journalctl -u api --since "10 min ago" -p err --no-pager | tail -20
```

JSON logs

```
jq -r 'select(.level=="error") | [.ts, .request_id, .msg] | @tsv' app.log | head
jq -r '.path' app.log | sort | uniq -c | sort -nr | head
jq -s 'map(select(.duration_ms)) | map(.duration_ms) | sort | .[length*95/100|floor]' app.log
```

`select` filters, `@tsv` makes tabular text and `-s` reads the whole file as an array for aggregate math.

Streaming

```
tail -F /var/log/app/app.log | jq -r --unbuffered 'select(.level=="error") | .msg'
zgrep -h "request_id=abc123" /var/log/app/app.log*.gz
```

Use `-F` so the tail survives log rotation. Save useful pipelines as small scripts with arguments, and test them on a sample before running on huge files.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How Log analysis one-liners and pipelines fits together

Part	What it does	What goes wrong
Count and rank	The pattern is extract, sort, count, rank.	Using uniq without sort.
Time windows and latency	.	Check the Common mistakes list below this diagram for what usually goes wrong here.
JSON logs	select filters, @tsv makes tabular text and -s reads the whole file as an array for aggregate math.	Parsing JSON logs with grep instead of jq.
Streaming	Use -F so the tail survives log rotation.	Running heavy pipelines on production hosts unthrottled.

Flow: Count and rank to Time windows and latency / Time windows and latency to JSON logs / JSON logs to Streaming

Incident: Top failing endpoint in a minute

Incident: *A four-command pipeline located the failing endpoint during an outage.*

```
$ awk '$9>=500{print $7}' access.log | sort | uniq -c | sort -nr | head -3
4120 /api/checkout
188 /api/cart
12 /healthz
```

The error rate had risen, and the dashboard was slow. The pipeline showed that almost all errors came from one endpoint, which pointed the team at a recent deploy of the checkout service. They rolled back and finished the review afterwards.

Common mistakes

- Using uniq without sort
- Running heavy pipelines on production hosts unthrottled
- Parsing JSON logs with grep instead of jq
- Forgetting log rotation when tailing

Check yourself

Why sort before uniq -c?

Answer: uniq counts only adjacent duplicates

Which tool parses JSON logs well?

Answer: jq

Why use tail -F?

Answer: It follows files across rotation

What is a good pipeline pattern?

Answer: Extract, sort, count, rank

MARGA by EknathaLabs
marga.eknathalabs.com

Cheat sheet: Bash scripting

Safe script headers, variables, conditionals, loops, functions and text-processing one-liners.

Safe header

```
#!/usr/bin/env bash
set -euo pipefail
IFS=$'\n\t'
```

Option	Effect
`-e`	Exit when a command fails
`-u`	Treat unset variables as errors
`-o pipefail`	A pipeline fails if any stage fails

Variables and quoting

```
name="web"
echo "Hello, $name"              # double quotes expand variables
echo 'Literal $name'             # single quotes do not
echo "${name:-default}"          # default if unset or empty
echo "${#name}"                  # length
echo "${path##*/}"               # strip longest prefix up to /
echo "${file%.txt}"              # strip suffix
readonly LIMIT=10
```

Always quote expansions: `"$file"`, `"$@"`.

Conditionals

```
if [[ -f "$file" && -r "$file" ]]; then
  echo "readable file"
elif [[ -d "$file" ]]; then
  echo "directory"
else
  echo "missing"
fi
```

Test	True when
`-f F` / `-d F`	F is a file / directory
`-s F`	F exists and is not empty
`-z S` / `-n S`	String is empty / not empty
`A == B`	Strings equal (inside `[[ ]]`)
`N -gt M`	Number greater than

Loops

```
for f in *.log; do
  [[ -e "$f" ]] || continue
  gzip "$f"
done

while IFS= read -r line; do
  echo "line: $line"
done < input.txt

for i in {1..5}; do echo "$i"; done
```

MARGA by EknathaLabs
marga.eknathalabs.com

Functions and arguments

```
usage() { echo "usage: $0 ENV" >&2; exit 2; }

main() {
  local env="${1:-}"
  [[ -n "$env" ]] || usage
  echo "deploying to $env"
}

main "$@"
```

Variable	Meaning
`$0`	Script name
`$1 ... $9`	Positional arguments
`$@`	All arguments, each separately quoted
`$#`	Number of arguments
`$?`	Exit status of the last command

Cleanup and temp files

```
tmp="$(mktemp -d)"
trap 'rm -rf "$tmp"' EXIT
```

Text processing

```
grep -E "ERROR|WARN" app.log | wc -l
awk '{print $1}' access.log | sort | uniq -c | sort -rn | head
awk -F, '$3 > 100 {print $1, $3}' data.csv
sed -i.bak 's/old/new/g' config.txt
cut -d: -f1 /etc/passwd
jq -r '.items[].metadata.name' pods.json
xargs -P4 -n1 gzip < filelist.txt
```

Handy patterns

```
command -v jq >/dev/null || { echo "jq required" >&2; exit 1; }
retry() { local n=0; until "$@"; do n=$((n+1)); [[ $n -ge 3 ]] && return 1; sleep $((2**n)); done; \
     }
exec 9>/tmp/job.lock && flock -n 9 || { echo "already running"; exit 1; }
```

Safer scripts

```
#!/usr/bin/env bash
set -Eeuo pipefail
IFS=$'\n\t'
trap 'echo "failed at line $LINENO" >&2' ERR
trap 'rm -rf -- "$tmp"' EXIT
tmp=$(mktemp -d)
: "${ENVIRONMENT:?set ENVIRONMENT}"
command -v jq >/dev/null || { echo "jq is required" >&2; exit 1; }
```

Warning: `set -e` *is not a full safety net. It does not stop on failures inside* `if` *conditions,* `&&` *lists or command substitutions in some cases. Check exit codes of the commands that matter.*

Danger: `rm -rf "$tmp"` *with an empty* `tmp` *is dangerous. The* `:?` *form (*`${tmp:?}`*) stops the script instead.*

Variables and expansion

```
name=${1:-default}
echo "${path##*/}"
echo "${file%.txt}.md"
echo "${#name}"
echo "${name^^}"
arr=(a b c); echo "${arr[@]}" "${#arr[@]}"
declare -A ages=([ann]=30 [bob]=25); echo "${ages[ann]}"
echo "$(date +%F)"
echo $((5 * 3))
printf '%s\n' "${arr[@]}"
```

MARGA by EknathaLabs marga.eknathalabs.com

Warning: *Always quote variables:* `"$var"`*. An unquoted value with spaces splits into several arguments, and* `*` *is expanded to file names.*

Loops, tests and functions

```
for f in *.log; do [ -e "$f" ] || continue; gzip "$f"; done
while IFS= read -r line; do echo "$line"; done < hosts.txt
[[ -f file && -r file ]] && echo readable
[[ $x =~ ^[0-9]+$ ]] && echo number
case "$1" in start) run ;; stop) halt ;; *) usage ;; esac
retry() { local n=0; until "$@"; do ((++n >= 5)) && return 1; sleep $((2**n)); done; }
```

Input, output and files

```
cmd > out.txt 2> err.txt
cmd > all.txt 2>&1
cmd | tee out.txt
cmd <<'EOT'
literal text, no expansion
EOT
exec 3>>audit.log; echo "ran" >&3
mapfile -t lines < file.txt
find . -name '*.tmp' -print0 | xargs -0 rm -f
flock -n /tmp/job.lock ./job.sh
```

Danger: `>` *truncates the file immediately. A typo such as* `cmd > important.conf` *destroys the content. Use* `set -o noclobber` *or* `>>` *to append.*

Warning: `eval` *runs a string as code. Never pass user input to it. Prefer arrays and* `"${cmd[@]}"`*.*

Danger: `xargs rm` *and* `find -delete` *act at once. Run the command with* `-print` *or* `echo` *first.*

Debugging

```
bash -n script.sh
bash -x script.sh
set -x; risky; set +x
shellcheck script.sh
PS4='+ ${BASH_SOURCE}:${LINENO}: ' bash -x script.sh
echo "${PIPESTATUS[@]}"
```

Warning: `set -x` *prints every command, including ones that contain passwords or tokens. Turn it off around secrets, and do not leave it on in CI logs.*

MARGA by EknathaLabs
marga.eknathalabs.com